

bdd Contributed Examples (contrib/)

This directory provides practical, ready-to-run examples demonstrating how to use **bdd** as both a **CLI tool** and a **native library (C-ABI and Python)** for real-world binary formats.

```
contrib/
├─ Makefile                # Builds C binaries and executes all tests
├─ README.md               # This documentation
├─ README.pdf              # Print-ready vector PDF documentation
├─ data/                   # Realistic binary sample files
│   └─ sample.mp3          # MPEG-1 Layer III audio frames (128k & 160k)
│   └─ sample.ts           # MPEG-2 Transport Stream (188B packets with PIDs 0, 256, 257)
│   └─ sample.wav          # RIFF WAVE 16-bit stereo PCM audio (44.1 kHz)
│   └─ sample_24bit.raw    # Raw 24-bit studio signed PCM audio
│   └─ sample.mp4          # ISO-BMFF MP4 with ftyp, mdat (H.264 NALs), and moov
│   └─ sample.jpg          # JPEG JFIF with SOF0 (640x480, 4:2:0 subsampling)
│   └─ sample.safetensors  # Safetensors file with FP8, BF16, and FP16 tensors
│   └─ sample_nvfp4.bin    # NVIDIA Blackwell NVFP4 (E2M1) 4-bit packed weights
│   └─ sample_fp6.bin      # OCP Microscaling FP6 (E3M2) 6-bit unaligned weights
├─ shell/                  # Standalone shell scripts using the bdd CLI
│   └─ decode_mp3.sh       # Unaligned 32-bit MP3 frame header parsing & frame seeking
│   └─ decode_mpeg_ts.sh   # MPEG-TS 13-bit PID inspection & container striding
│   └─ decode_wav.sh       # WAV header parsing, stereo demuxing & 24-bit downsampling
│   └─ decode_mp4.sh       # MP4 box traversal & H.264 NAL bitfield extraction (1U2U5U)
│   └─ decode_jpeg.sh      # JPEG SOF0 geometry & 4-bit chroma nibble extraction (4U4U)
│   └─ decode_ai_weights.sh # Safetensors 0(1) seeking & NVFP4/FP6/FP8/BF16 decoding
│   └─ run_all_shell_examples.sh # Master script running all shell demonstrations
├─ python/                 # Python scripts using libbdd ctypes bindings
│   └─ generate_samples.py  # Deterministic generator for all data/ sample files
│   └─ decode_media.py      # MP3, MPEG-TS, JPEG, and H.264 NAL parsing
│   └─ decode_ai_weights.py # Safetensors inspection, NVFP4, and FP6 decoding
└─ c/                      # Native C programs linking against libbdd.so
    └─ decode_media.c       # bdd_unpack_u64, bdd_pack_u64, and bit reversal
    └─ decode_ai_weights.c  # Native FP4, FP6, FP8, BF16, and FP16 float decoders
```

Quick Start

Run All Demonstrations

From the repository root:

```
make contrib
```

Or from inside `contrib/` :

```
cd contrib
make test
```

Regenerate Sample Data Files

To recreate all test binary files from scratch:

```
python3 contrib/python/generate_samples.py
```

Format & Bitstream Reference

1. MPEG-1 Layer III Audio (MP3) Frame Headers

Standard Unix tools cannot inspect MP3 frame headers because fields do not align to byte boundaries:

- **Header Bitfield (32 bits = 4 bytes):** `11U` (Syncword `0x7FF`) `2U` (MPEG Version) `2U` (Layer) `1U` (CRC Protection) `4U` (Bitrate Index) `2U` (Sample Rate Index) `1U` (Padding) `1U` (Private) `2U` (Channel Mode) `2U` (Mode Extension) `1U` (Copyright) `1U` (Original) `2U` (Emphasis)
- **CLI Pattern:**

```
bdd --input-pattern="110202010402010102020101020" --count=1 --output-json < sample.mp3
```

- **$O(1)$ Frame Seeking:**

```
# Jump directly to frame 2 at byte 417:
bdd --input-file=sample.mp3 --input-skip-bits=417B --input-
pattern="110202010402010102020101020" --count=1 --output-json
```

2. MPEG-2 Transport Stream (MPEG-TS)

MPEG-TS uses repeating 188-byte containers (1504 bits). The 4-byte header contains an unaligned **13-bit Packet Identifier (PID)**:

- **Header Pattern:** `801010101302020401472x` (Sync `0x47`, TEI, PUSI, Priority, **13-bit PID**, Scrambling, Adaptation, Continuity Counter, plus 184-byte payload skip).
- **Container Stride & Offset:**

```
# Extract only the 13-bit PID from every 188-byte packet:
bdd --input-raw-unit=1504 --input-offset=11 --input-unit=13 --output-unit=13 --output-integers < sample.ts
```

3. WAV / RIFF Audio & Multi-Channel Demuxing

- **Stereo Demuxing:** Demux interleaved 16-bit stereo PCM into two independent mono audio files in a single pass:

```
bdd --input-file=sample.wav --input-skip-bits=352 --input-little-endian \
    --input-pattern="16S16S" --demux-files="left.raw,right.raw"
```

- **24-bit PCM Downsampling:** Read non-standard 24-bit signed integers (24S), right-shift 8 bits, and output 16-bit signed PCM:

```
bdd --input-file=sample_24bit.raw --input-pattern="24S" --remove-right=0,8 --output-unit=16 --output-tuples
```

4. MP4 / ISO-BMFF & H.264 NAL Units

- **MP4 Box Walking:** Inspect nested 32-bit big-endian length and 4-byte FourCC ASCII box headers:

```
bdd --input-pattern="32U32C" --count=1 --output-json < movie.mp4
```

- **H.264 NAL Unit Headers:** Decode 1U (Forbidden Zero), 2U (NAL Reference IDC), 5U (NAL Unit Type: SPS=7, PPS=8, IDR=5, Slice=1):

```
bdd --input-skip-bits=36B --input-pattern="32U1U2U5U" --output-json < movie.mp4
```

5. JPEG SOF0 Chroma Subsampling Nibbles

In baseline JPEG (0xFFC0), the horizontal and vertical chroma subsampling factors (e.g., 2×2 for 4:2:0) are stored as **4-bit nibbles** (4U4U):

- **Pattern with Repetition Multiplier:**

```
bdd --input-skip-bits=24B --input-pattern="8U16U16U8U3*(8U4U4U8U)" --count=1 --output-json < sample.jpg
```

6. AI Model Weights (Safetensors, NVFP4, FP6, FP8, BF16)

`bdd` natively decodes sub-byte and modern floating-point formats:

CODE	FORMAT	BIT WIDTH	DESCRIPTION
4E / 4e	NVFP4 (E2M1)	4 bits	NVIDIA Blackwell / OCP Microscaling FP4
6E / 6e	FP6 (E3M2)	6 bits	OCP Microscaling FP6
8E / 8e	FP8 (E4M3FN)	8 bits	NVIDIA Hopper H100 / OCP FP8
8Q / 8q	FP8 (E5M2)	8 bits	NVIDIA Ada Lovelace / OCP FP8
16Y / 16y	BF16	16 bits	Google Brain Bfloat16
16H / 16h	FP16	16 bits	IEEE 754 Half-Precision Float

- **Safetensors Inspection:**

```
# 1. Read 64-bit LE JSON header length:
HEADER_LEN=$(bdd --input-pattern="8*8U" --count=1 --output-json < model.safetensors | jq
'.[0] + .[1]*256 + .[2]*65536 + .[3]*16777216')
# 2. Seek directly to tensor offset in O(1) time and unpack FP8 weights:
bdd --input-skip-bits=$(( 8 + HEADER_LEN ))B --input-pattern=8E --output-json <
model.safetensors
```

- **NVIDIA Blackwell NVFP4 (Sub-Byte 4E4E):**

```
bdd --input-pattern="4E4E" --output-json < sample_nvfp4.bin
```

- **OCP FP6 (Unaligned 4*6E across 24 bits):**

```
bdd --input-pattern="4*6E" --output-json < sample_fp6.bin
```

Native C & Python APIs

C API (`include/bdd.h` & `libbdd.so`)

```
#include "bdd.h"

// Unpack unaligned bitfields
uint64_t fields[4];
bdd_unpack_u64("8U4U4U8U", comp_u24, fields, 4);

// Convert AI floating point numbers
double w0 = bdd_decode_fp4_e2m1(0x03); // 1.50
double fp8 = bdd_decode_fp8_e4m3(0x38); // 1.00
double bf16 = bdd_decode_bf16(0x3F80); // 1.00
```

Python API (`python/bdd.py`)

```
from bdd import Bdd

b = Bdd()
# Unpack bitfields
fields = b.unpack("11U2U2U1U4U2U1U1U2U2U1U1U2U", mp3_header_u32)

# AI float codecs
val_fp4 = b.decode_fp4(0x03)
val_fp8 = b.decode_fp8(0x38)
val_fp6 = b.decode_fp6(0x10)
```